

SHIVI X100+

Research-Grade · Production-Grade · AI-Native Enterprise Operating Platform

A unified ecosystem combining a shared kernel and platform fabrics with 100 production and research systems — purpose-built for the autonomous enterprise. Not 100 disconnected applications. One operating platform.

TARGET ARCHITECTURE

EXECUTIVE & TECHNICAL BRIEFING

FOUNDERS · ENGINEERS · INVESTORS



The Problem: The Enterprise is Drowning in Disconnected Intelligence

The Status Quo Is Broken

Modern enterprises run on a patchwork of SaaS tools, point solutions, and bespoke integrations — each generating data, each requiring separate identity models, each with its own authorization logic, observability story, and operational overhead. The result is an AI deployment landscape that is fragmented by design.

AI copilots are bolted onto existing applications without shared memory, shared context, or shared policy. Agents cannot trust one another. Tools cannot be governed at runtime. Models propose actions with no platform-level authorization layer to verify intent, validate outputs, or capture evidence.

Identity Fragmentation

No unified identity primitive across agents, users, tools, and tenants

Policy Blindness

No runtime authorization layer; agents act with unconstrained delegation

Context Amnesia

No shared context, memory, or knowledge plane across systems

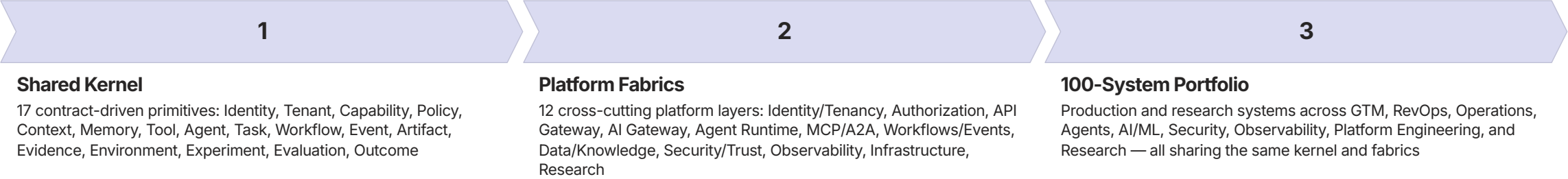
Evidence Vacuum

No provenance, audit trail, or outcome traceability across AI actions

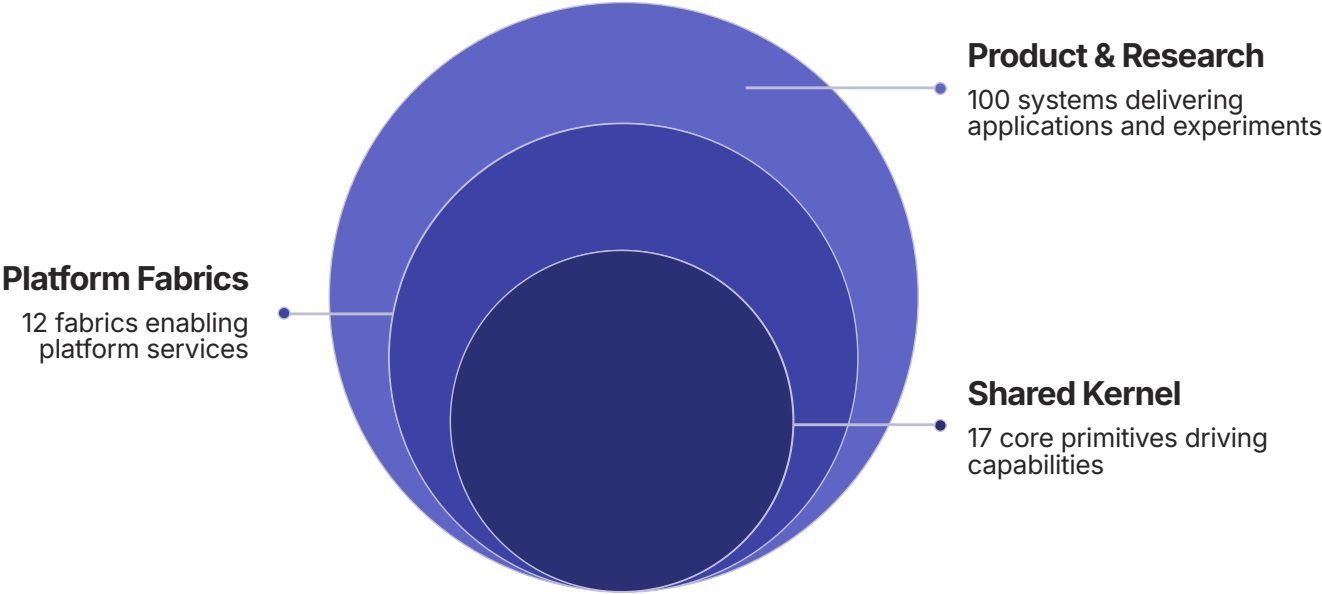
⚠ The enterprise AI gap is not a model quality problem. It is a platform problem. ShiVi is the platform.

The ShiVi Thesis: One Kernel. One Platform. 100 Systems.

ShiVi is not a suite of AI features added to enterprise software. It is a ground-up re-architecture of how enterprise operating platforms are built — with AI, agents, and autonomous workflows as first-class kernel primitives from day one.

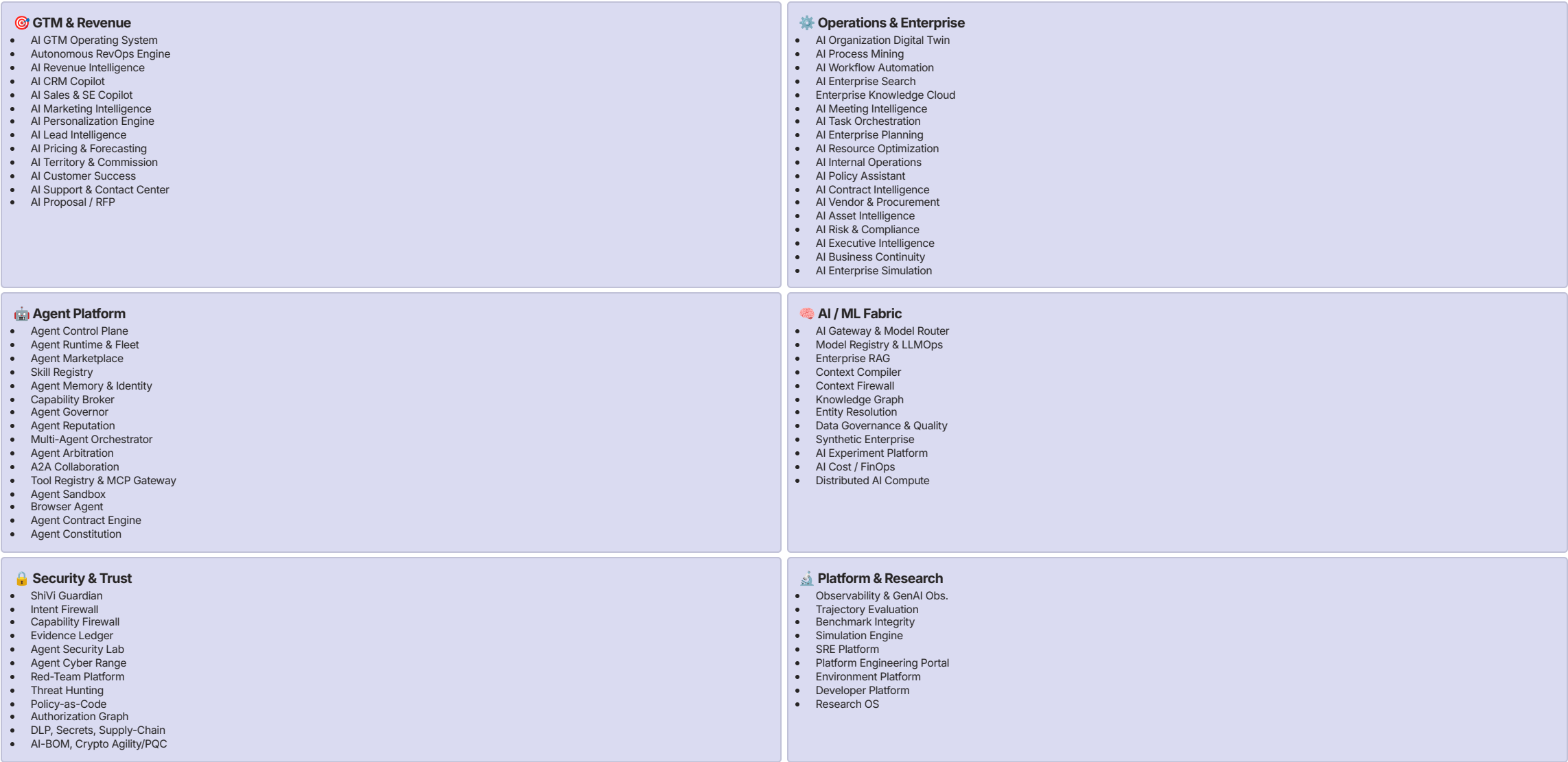


The principle is architectural unification: every system in the portfolio inherits identity, policy, context, observability, and security from the shared platform rather than re-implementing them. **Model proposes. ShiVi authorizes.** This single constraint reshapes the entire risk posture of enterprise AI deployment.



The 100-System Ecosystem

ShiVi's portfolio spans six major domains. Every system is built on the shared kernel — none re-implements identity, policy, context, or observability from scratch.



Kernel Primitives & Agent Execution Contract

17 Kernel Primitives

Every ShiVi system operates on a shared set of contract-driven, versioned primitives. These are not abstractions — they are enforced runtime types with schemas, lifecycle state machines, and policy hooks.

Identity

Capability-bound, SPIFFE/SPIRE-attested identities for users, agents, tools, and services

Tenant

Full multi-tenant isolation across API, DB, cache, vector, graph, events, storage, agents, and billing

Policy

OPA/OpenFGA/Kyverno enforced at runtime; desired-state reconciliation with audit evidence

Context

Versioned, provenance-tracked context packets with quality scoring and firewall enforcement

Memory

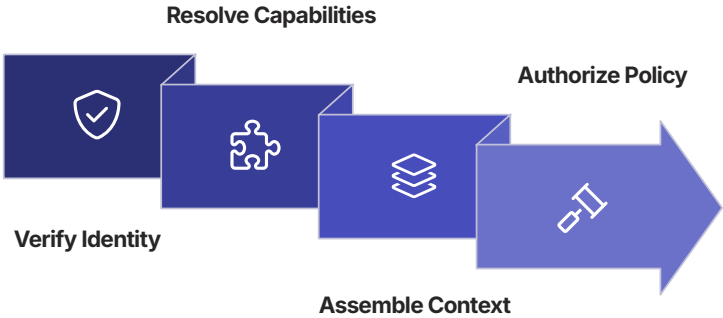
Governed agent memory with poisoning defenses, epoch management, and selective retention

Evidence

Immutable provenance ledger:
UNKNOWN → DISCOVERED →
PLANNED → IMPLEMENTED →
TESTED → VERIFIED →
PRODUCTION_READY

Agent Execution Flow

Every agent action traverses a mandatory authorization and verification pipeline. No agent bypasses this flow — it is enforced at the kernel level, not by convention.



❏ **Core constraint:** Model proposes. ShiVi authorizes. Risk-tiered autonomy gates high-impact actions behind human approval workflows with full evidence capture.

Security, Trust & Zero-Trust Agent Governance

ShiVi's security model is not perimeter-based. It is a zero-trust, policy-enforced, evidence-producing mesh that governs every agent action, every tool invocation, and every model output at runtime.



Identity & Access

OIDC/OAuth, SPIFFE/SPIRE workload identity, OpenFGA authorization graph, OPA policy engine, Kyverno admission control. Capability-bound identities prevent privilege escalation and cross-tenant access at the cryptographic layer.



Supply-Chain & Provenance

Sigstore/Cosign artifact signing, Trivy vulnerability scanning, SBOM and AI-BOM generation, Cilium/Hubble network policy, Secrets management, DLP enforcement, and crypto agility with Post-Quantum Cryptography readiness.

Threat surface addressed: Prompt injection, indirect injection, tool abuse, memory poisoning, context poisoning, privilege escalation, cross-tenant access, malicious MCP/A2A servers, unsafe delegation, data exfiltration, and supply-chain compromise. High-assurance mode and confidential execution profiles available for regulated workloads.



Tri-Layer Agent Firewall

Intent Firewall detects prompt injection and unsafe delegations before execution. **Capability Firewall** enforces least-privilege tool access at runtime. **Context Firewall** prevents memory/context poisoning and untrusted content injection.



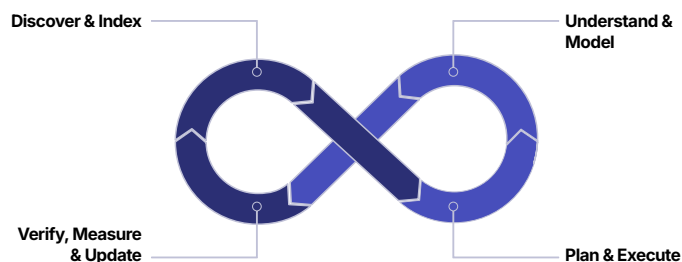
Evidence Ledger & Auditability

Every agent action, policy decision, tool invocation, and model output produces an immutable evidence record. The Evidence Ledger supports compliance audits, incident reconstruction, and regulatory reporting with full provenance chains.

Context Loop MAX & AI / RAG Architecture

Context Loop MAX

Context engineering is the core differentiator of ShiVi's AI fabric. The platform does not blindly append conversation history or retrieved documents. It runs a 12-phase context lifecycle that produces a scored, verified, budget-constrained Context Packet before any model receives input.



Each Context Packet carries: source provenance, freshness scores, trust tier classification, compression metadata, epoch markers, and a Context Quality Score. External and retrieved content is treated as untrusted by default and passes through the Context Firewall before assembly.

Enterprise RAG & AI Gateway

The AI Gateway and Model Router sit in front of all model invocations, providing unified routing, cost attribution, rate limiting, fallback, and observability. The Enterprise RAG pipeline layers context engineering on top of vector, graph, and full-text retrieval.

01

Query Understanding

Intent classification, entity extraction, query decomposition and sub-question generation

02

Multi-Source Retrieval

Vector DB, Knowledge Graph (Neo4j), OpenSearch full-text, ClickHouse analytics, and structured DB retrieval in parallel

03

Context Assembly

Collect → Classify → Filter → Rank → Deduplicate → Compress → Verify → Budget → Assemble with Context Firewall enforcement

04

Model Invocation

AI Gateway routes to optimal model; ShiVi authorizes the plan; model executes within capability bounds

05

Evaluation & Evidence

Trajectory evaluation, hallucination detection, RAG quality scoring, and evidence capture before outcome is committed

Platform Fabrics: Data, Workflows, Events & Observability

Data Architecture

ShiVi's data plane is polyglot by design — each storage tier is selected for its access pattern, not for uniformity. All tiers enforce multi-tenant isolation at the query, schema, and network layer.

Tier	Technology	Purpose
Transactional	PostgreSQL (RLS)	Canonical entities, contracts, policies, approvals
Cache / State	Redis / Valkey	Session, rate limit, feature flags, hot context
Vector / Semantic	pgvector / Qdrant	Embeddings, RAG retrieval, semantic search
Graph	Neo4j	Knowledge graph, authorization graph, entity resolution
Search	OpenSearch	Enterprise search, log aggregation, full-text
Analytics	ClickHouse	Observability metrics, cost/FinOps, evaluation data
Lakehouse	dbt + object storage	Data warehouse, ML training, compliance archival
Streaming	NATS / Kafka / Redpanda	Versioned, idempotent events with full correlation IDs

Workflow, Event & Observability Plane

All durable workflows run on Temporal with full tenant, actor, correlation, causation, and trace ID propagation. Events are versioned, idempotent, and schema-validated at publish time.

Temporal Workflows

Long-running, durable, retryable workflows with saga compensation, human approval gates, and full event replay

Event Bus

NATS/Kafka events carry tenant, actor, correlation, causation, and trace IDs. Schema registry enforces backward-compatible versioning

OpenTelemetry Traces

End-to-end traces: User → API → Agent → Context → RAG → Model → Tool → Workflow → Policy → Data → Outcome

SLO & Error Budgets

Per-service SLO definitions, error budget burn alerts, dependency graphs, change intelligence, and chaos engineering integration

- ❑ Canonical entities — Tenant, User, Organization, Account, Lead, Opportunity, Contract, Invoice, Agent, Tool, Model, Memory, Policy, Incident, Experiment — are versioned across all storage tiers.

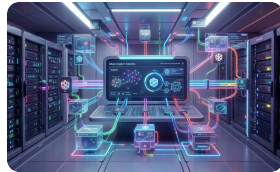
Platform Engineering, Developer Experience & Research OS

ShiVi treats the developer platform as a first-class product. Golden paths eliminate toil and enforce standards without restricting autonomy. Every service bootstrapped through the platform portal is production-ready by default.



Golden Paths

Automated scaffold generation: repo structure, Dockerfile, test harnesses, OpenTelemetry instrumentation, SBOM generation, security scanning, CI/CD pipeline, policy definitions, SLO configuration, Grafana dashboards, runbooks, and rollback procedures — in one command.



Infrastructure & GitOps

Backstage developer portal with software catalog and TechDocs. Crossplane for infrastructure-as-code. Argo CD for GitOps reconciliation. Argo Rollouts for progressive delivery with canary analysis. OpenFeature for feature flag management. Environments span: local → dev → test → integration → staging → preview → canary → production → research → high-assurance.



Research OS & Innovation Pipeline

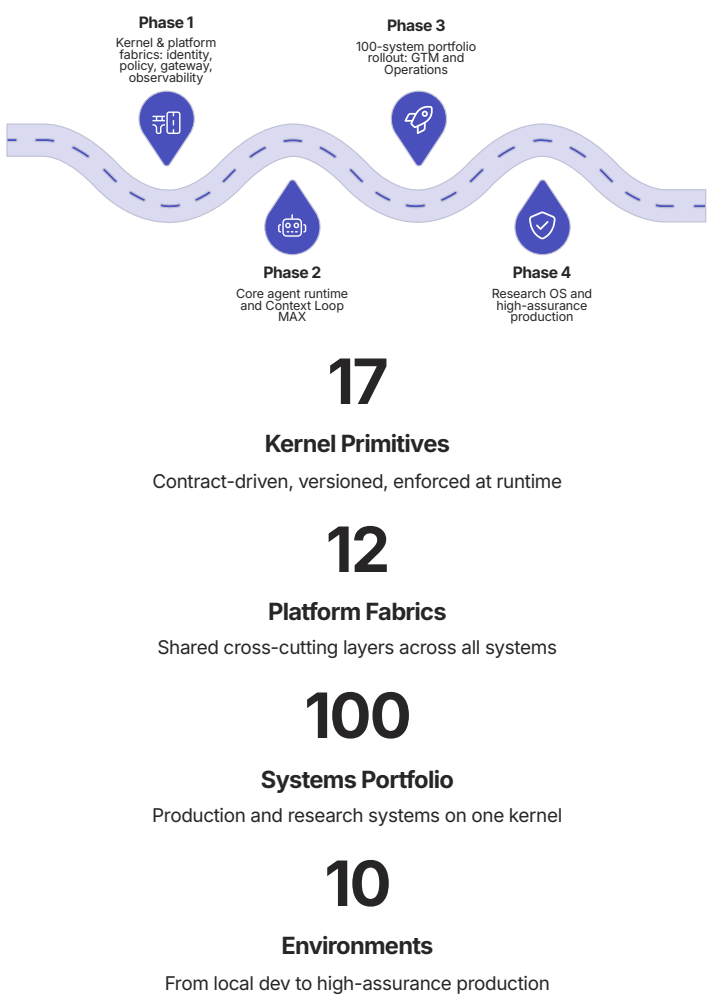
Trajectory evaluation, benchmark integrity, adversarial probes, synthetic enterprise generator, cyber range, simulation engine, counterfactual decision engine, and digital twin. Research-to-production promotion follows: Experiment → Evaluation → Security Review → Canary → Approval → Production. No uncontrolled self-modification. All model, agent, prompt, tool, and data versions are pinned and reproducible.

Technology Stack, Delivery Roadmap & KPIs

Core Technology Ecosystem

Frontend Next.js · React · TypeScript · Design tokens · AI-native UX · Streaming · Command palette · Accessibility · Visual regression	Backend Node.js 24 LTS · Python · Fastify / NestJS · Zod / JSON Schema · Temporal · NATS / Kafka / Redpanda · MCP · A2A
AI / ML ShiVi AI Gateway · vLLM · KServe · MLflow · Ray · LLMOps · RAG · Context Compiler · Knowledge Graph	Security OpenFGA · OPA · Kyverno · OIDC/OAuth · SPIFFE/SPIRE · Cilium/Hubble · Sigstore/Cosign · Trivy · PQC
Observability OpenTelemetry · Prometheus · Grafana · Loki · Tempo / Jaeger · Trajectory Evaluation · Benchmark Integrity	Platform Kubernetes · Gateway API · Backstage · Crossplane · Argo CD · Argo Rollouts · OpenFeature · Terraform / OpenTofu

Delivery Phases & Readiness KPIs



☐ **Evidence states** govern all delivery claims: UNKNOWN → DISCOVERED → PLANNED → IMPLEMENTED → TESTED → VERIFIED → PRODUCTION_READY. Target architecture is clearly distinguished from verified implementation at all times.